

SAMATRIX RESUMEFORGE 2026

RESUME CLASSIFICATION — PARTICIPANT APPROACH GUIDE

A practical end-to-end NLP workflow:

Problem → Data → EDA → Preprocessing → Features → ML/DL → Evaluation → Demo

IMPORTANT: Do not jump directly to model training. Your solution should show that you understood the resume text, identified the data problems, justified preprocessing and feature choices, built meaningful baselines, and compared models.

1. Problem Statement

Build an end-to-end Resume Classification system using the provided resume dataset. Given a resume (raw text or extracted text), predict its appropriate resume category/class. The final solution should include data understanding, exploratory data analysis (EDA), text preprocessing, feature engineering, model training, evaluation, error analysis, and a simple prediction interface or demo.

2. What We Expect from Your Approach:

- Understand the dataset before modelling: class distribution, text length, missing values, duplicates, noisy text, and class-specific vocabulary.
- Show EDA visually. At minimum: class distribution, text-length distribution, most frequent words, and n-gram analysis. WordCloud is useful but should not be your only EDA.
- Apply preprocessing deliberately. Do not remove information blindly; resume text contains useful skills, technologies, certifications, education terms, job titles, and experience indicators.
- Build at least one strong classical ML baseline using TF-IDF.
- Build at least one neural/deep-learning approach. Word2Vec is a representation technique, not a classifier by itself; combine embeddings with an appropriate model such as LSTM/GRU/Dense network.
- Compare models using appropriate metrics rather than selecting a model only because its accuracy is high.
- Perform error analysis: inspect wrongly classified resumes and explain why the model fails.
- Make the final pipeline reproducible: preprocessing used during training must be identical during prediction.

3. Recommended End-to-End Flow

Step	Stage	What You Should Do	Marks
1	Problem Understanding	Define input, output, classes, constraints and success metric.	5
2	Data Gathering	Load dataset; inspect files, columns, labels and raw resume text.	5
3	Data Quality Check	Missing values, duplicates, corrupted/empty resumes, label inconsistencies, leakage.	5
4	EDA	Class distribution, text length, vocabulary, WordCloud, unigram/bigram/trigram analysis.	10
5	Text Preprocessing	Normalize text carefully; remove noise while preserving meaningful resume information.	10
6	Train/Validation/Test Split	Split before fitting vocabulary/vectorizers to prevent data leakage.	5
7	Feature Engineering	TF-IDF baseline; optionally Word2Vec/embeddings; derive useful text statistics.	5
8	ML Models	Logistic Regression / Linear SVM / Naive Bayes or another justified baseline.	5
9	DL Model	Embedding/Word2Vec + LSTM/GRU/Dense model, or another justified neural approach.	5
10	Evaluation	Accuracy + Precision/Recall/F1, confusion matrix, per-class results.	5
11	Error Analysis	Inspect wrong predictions and identify data/model weaknesses.	5
12	Final Pipeline & Demo	Raw resume → preprocessing → feature extraction → model → predicted category.	5

Total: 70 Marks

4. EDA: The Most Important Part of This Challenge

Your EDA should answer: What does a resume look like in this dataset? How are classes distributed? What words and phrases distinguish categories? Where is the data noisy or problematic?

EDA	What to Plot/Calculate
Class distribution	Bar chart of resume categories
Resume length	Characters, words and/or tokens per resume
Word frequency	Top 20–30 words after sensible cleaning
Word Cloud	Overall and preferably class-wise
N-grams	Top unigrams, bigrams and trigrams
Class-wise vocabulary	Top TF-IDF terms or frequency per class
Missing/empty text	Counts and percentages
Duplicates	Exact duplicate count

5. Text Preprocessing

Create a reproducible preprocessing function. Apply the same transformation during training and inference.

- Convert text to a consistent case where appropriate.
- Normalize whitespace and line breaks.
- Remove HTML, unwanted markup, repeated spaces and obvious encoding artifacts if present.
- Handle URLs, email addresses and phone numbers according to the problem. Do not automatically delete every symbol if it carries useful information.
- Tokenize when required by the selected model.
- Remove stopwords only after testing their impact. For TF-IDF, stopwords removal is optional; it is not a universal rule.
- Lemmatization or stemming: compare the effect rather than assuming it always improves performance.
- Preserve meaningful technical tokens such as Python, C++, SQL, AWS, TensorFlow, Java, .NET, NLP, etc.
- Consider preserving section-related information such as skills, education, experience, certifications and job titles if the dataset structure allows it.
- Remove empty/very short documents only with a justified rule.
- Do not fit vocabulary, IDF statistics, Word2Vec training, or scalers on the test set.

6. Feature Engineering

A. TF-IDF — Required Baseline

- Use TfidfVectorizer for a strong and interpretable baseline.
- Experiment with ngram_range such as (1,1) versus (1,2).
- Tune min_df/max_df and max_features if the vocabulary is very large.
- Compare raw word TF-IDF with word + bigram TF-IDF.
- Inspect top weighted terms for each class to understand what the model learned.

B. Word2Vec — Neural Representation

- Word2Vec converts words into dense vectors based on contextual usage in the corpus.
- Use pretrained Word2Vec if available, or train Word2Vec on the training corpus only.
- A document needs a document-level representation: e.g., mean/max pooling of word vectors, or sequence embeddings fed into an LSTM/GRU.
- Do not claim 'Word2Vec is the model'—it is a representation/embedding method.

7. Model Selection: you can use any model.

Model	Input Representation	Role
Multinomial Naive Bayes	TF-IDF / count features	Fast baseline
Logistic Regression	TF-IDF	Primary classical baseline
Linear SVM	TF-IDF	Strong text-classification baseline
LSTM / GRU	Token sequences / embeddings	Deep-learning model
Dense NN with pooled Word2Vec	Word2Vec document vectors	Simple neural comparison
Transformer (optional)	Transformer tokenizer/embeddings	Advanced model

8. Train/Validation/Test Strategy

- Use a stratified split for classification when possible.
- A practical starting point is 70/15/15 or 80/20, with the training portion split again into validation if required.
- Keep the test set untouched until final comparison.
- Fit TF-IDF only on training data and transform validation/test data with the fitted vectorizer.
- Do not tune hyperparameters repeatedly against the test set.
- For imbalanced classes, consider class_weight='balanced' where supported and/or stratified sampling.

9. Evaluation

- Accuracy: useful but can hide poor performance on minority classes.
- Precision: among predicted samples for a class, how many were correct?
- Recall: among actual samples of a class, how many were found?
- F1-score: balances precision and recall.
- Macro-F1: gives equal importance to each class; important for imbalanced multiclass problems.
- Weighted-F1: accounts for class support.
- Confusion matrix: identify which resume categories are being confused.
- Per-class metrics: never report only one overall number.

10. Error Analysis

- Collect incorrect predictions from the validation/test set.
- Display: actual class, predicted class, confidence/decision score if available, and a shortened text preview.
- Look for overlapping categories, generic resumes, noisy text, very short resumes and mislabeled examples.
- Ask: Is the error caused by preprocessing, insufficient training data, class overlap, feature representation, or the model?
- Use these findings to justify your next improvement. Do not randomly keep changing models.

11. Final Prediction Pipeline

- Save the complete fitted preprocessing/vectorization pipeline and model.
- Ensure inference uses exactly the same preprocessing and feature order as training.
- Test the final system with at least 3–5 unseen resume examples.
- A simple Streamlit/Flask/FastAPI interface is a plus, but it should not replace the core ML/NLP work.